

Non-throwing container operations

Abstract

This paper explores alternatives for adding non-throwing container operations, namely alternatives to throwing exceptions from failing modifications.

There are various embedded environments where exceptions are not palatable. Yet the applications written for such environments are getting ever more complex, and C++ and its abstraction mechanisms are increasingly attractive on such environments. There's an impedance mismatch between the desire to have the environment be as close to standard C++ as possible and not being able to afford exception support.

Introduction

A fairly common example of the environments mentioned in the abstract are systems that have at most hundreds of kilobytes of RAM. Yet in that cramped space, device manufacturers put applications that have network connectivity, XML support, declarative UI frameworks with scripting, and more.

These environments often do ensure that the overall memory budget is not overrun by utilizing fixed-size pools. There are sometimes multiple pools so that individual module-specific pools cannot disturb each other, but the clients of the pools do recover from allocations failing to allocate memory from the pools.

Building without exception support for such resource-constrained systems allows cramming much more functionality into the system, from both the non-volatile memory point of view and the RAM point of view. The RAM consumption is often the more important point, as is the potentially non-deterministic handling time of thrown exceptions. There has been a lot of talk about making exceptions more palatable on such resource-constrained systems, but thus far that hasn't materialized in practice.

So what are you proposing?

In a very abstract sense, what we are proposing is adding alternative contained modifier operations of some form so that exceptions are not required for reporting errors. There are various ways in which that can be done, but certain general properties are probably common to all alternatives:

- Exception support costs too much, space-wise and time-wise.
- Dynamic memory allocations still happen, and they can fail.
- Not every module is critical, so various strategies can be employed to recover from allocation failures.
- Termination is not a feasible strategy.

Why not just terminate?

A fairly oft-suggested alternative is to just terminate if a failing allocation cannot throw. That's a very drastic approach, because many environments do not support multiple processes, so if any module terminates, the whole system will restart after a hard abort. This causes loss of system availability, which in some environments is life-threatening and in some environments loses user data.

Some proposal alternatives

Simple status return

Daniel Gutson's original proposal outline just adds non-throwing overloads to contained modifiers. Here's an example of what such overloads for vector's `push_back` would look like:

```
bool push_back(nothrow_t, const T&);  
bool push_back(nothrow_t, T&&);
```

The general idea is that we use a tag type to differentiate these overloads, and since the idea is that these operations don't throw if an allocation fails, we can just as well use the `nothrow_t` we already have. The function returns a `bool` signaling whether it succeeded or not. Note that the copy operation of `T` may throw; that throw is not handled by these functions. The intention is that these functions are used with an accompanying allocator that doesn't throw on failure, and the element type is one that has non-throwing copy/move operations. Also note that the idea is to add such non-throwing operations to other containers beyond `vector`.

Use a different function name instead

Various people suggested that since the behavior of these functions is significantly different from the other overloads, they should be named differently. As a strawman example, something like

```
bool push_back_nothrow(const T&);  
bool push_back_nothrow(T&&);
```

A much less intrusive alternative

Howard Hinnant pointed out that adding lots of new functions or overloads to the api of the library containers to be used by a specialized crowd is questionable. He outlined a possible alternative: allowing an allocator to return a `nullptr` and specifying the behavior in that case. For instance, a failed `push_back` just wouldn't change the size or the capacity of the vector.

A custom allocator with a failure callback

Howard outlined a different less intrusive approach: use an allocator that can register a failure callback. This is fairly similar to the previous approach in the sense that an allocator can fail without an exception and the result behavior of the container needs to be specified, but in addition to that, a user can set a failure callback on the allocator. That may end up being rather intrusive if the callback type is not erased. Erasing the type of the callback has its own problems.

Adding new specialized containers

Howard also suggested building a new container, since it's can be seen as a fair assessment that such a difference in behavior is significant enough that the facility should become a separate container.

What's next?

Before diving into detailed specification and implementation, design guidance is necessary. The whole approach needs to go to EWG, because the general idea of not using exceptions for error reporting is a completely new programming model for the library containers. Beyond that, the general design needs to be looked at by LEWG: whether to add overloads, whether to add new functions, whether to build new containers...